Домашнее задание № 18

1. Разворачиваем Superset
   - Создаем директорию superset и переходим в нее
   - Создаем упрощенную, но рабочую версию docker-compose.yml

docker-compose.yml – Блокнот

Файл Правка Формат Вид Справка

```
version: '3.8'

x-superset-image: &superset-image apache/superset:6.0.0
x-superset-depends-on: &superset-depends-on
  - db
  - redis

services:
  redis:
    image: redis:7
    container_name: superset-redis
    restart: unless-stopped
    volumes:
      - redis:/data
    networks:
      - clickhouse-network

  db:
    image: postgres:15
    container_name: superset-db
    restart: unless-stopped
    environment:
      POSTGRES_DB: superset
      POSTGRES_USER: superset
      POSTGRES_PASSWORD: superset
    volumes:
      - db_home:/var/lib/postgresql/data
    networks:
      - clickhouse-network

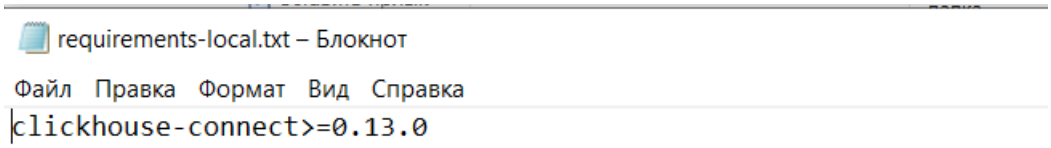
  superset:
    image: *superset-image
    container_name: superset
    restart: unless-stopped
    depends_on: *superset-depends-on
    environment:
      - SUPERSET_SECRET_KEY=change_me_please_to_random_string
      - SUPERSET_LOAD_EXAMPLES=no
    ports:
      - "8088:8088"
```

```
        ----------
    volumes:
      - ./docker/requirements-local.txt:/app/docker/requirements-local.txt
      - superset_home:/app/superset_home
    command: >
      /bin/sh -c "
      pip install -r /app/docker/requirements-local.txt &&
      superset db upgrade &&
      superset fab create-admin
        --username admin
        --firstname Admin
        --lastname Admin
        --email admin@example.com
        --password admin || true &&
      superset init &&
      /usr/bin/run-server.sh
      "
    networks:
      - clickhouse-network

volumes:
  db_home:
  redis:
  superset_home:

networks:
  clickhouse-network:
    external: true
    name: clickhouse-docker_clickhouse-network
```

- Создаем папку docker и файл requirements-local.txt с драйвером ClickHouse



- Запускаем Superset

```
docker compose up -d
```

- Проверяем HTTP-порт ClickHouse

```
$ winpty docker exec -it superset curl -s http://clickhouse-replica-1:8123/ping
Ok.
```

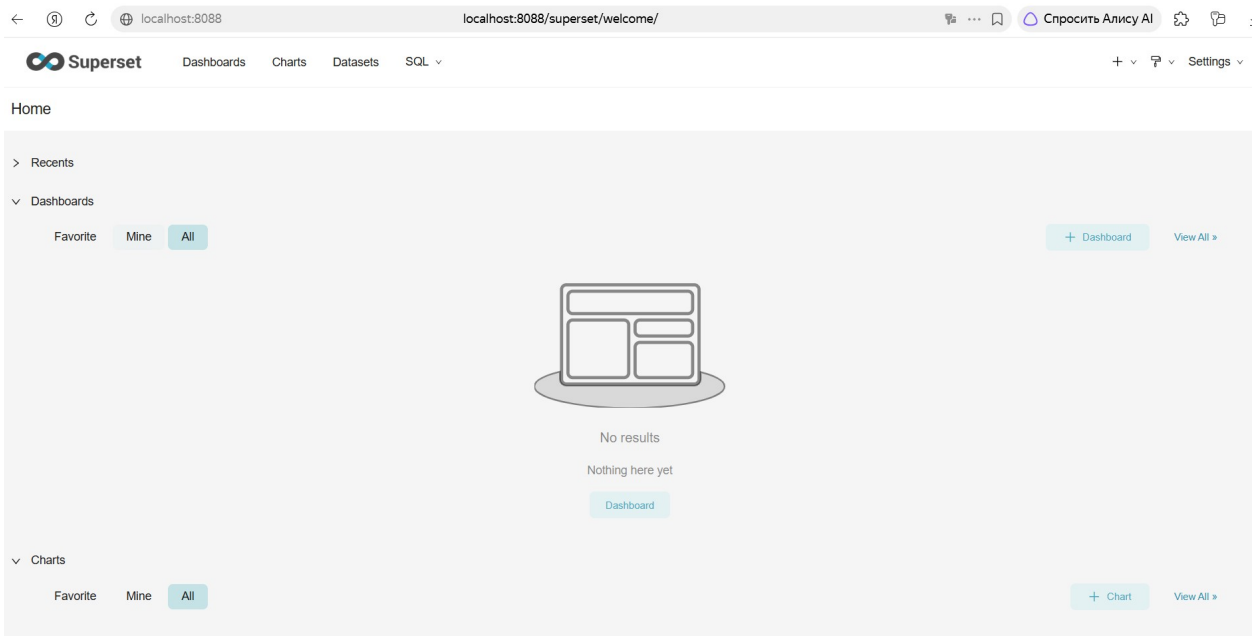
2. Запускаем Superset
- Создаем пользователя admin

```
$ winpty docker exec -it superset superset fab create-admin \
  --username admin \
  --firstname Admin \
  --lastname Admin \
  --email admin@example.com \
  --password admin
/app/.venv/lib/python3.10/site-packages/flask_limiter/extension.py:324: UserWarning: Using the in-memory storage for tracking
 rate limits as no storage was explicitly specified. This is not recommended for production use. See: https://flask-limiter.r
eadthedocs.io#configuring-a-storage-backend for documentation about configuring the storage backend.
  warnings.warn(
2026-10-12 20:56:14,017:INFO:alembic.runtime.migration:Context impl SQLiteImpl.
2026-10-12 20:56:14,017:INFO:alembic.runtime.migration:Will assume non-transactional DDL.
2026-10-12 20:56:14,249:INFO:superset.app:Syncing configuration to database...
2026-10-12 20:56:14,256:INFO:superset.app:Configuration sync to database completed successfully
Recognized Database Authentications.
Admin User admin created.
```

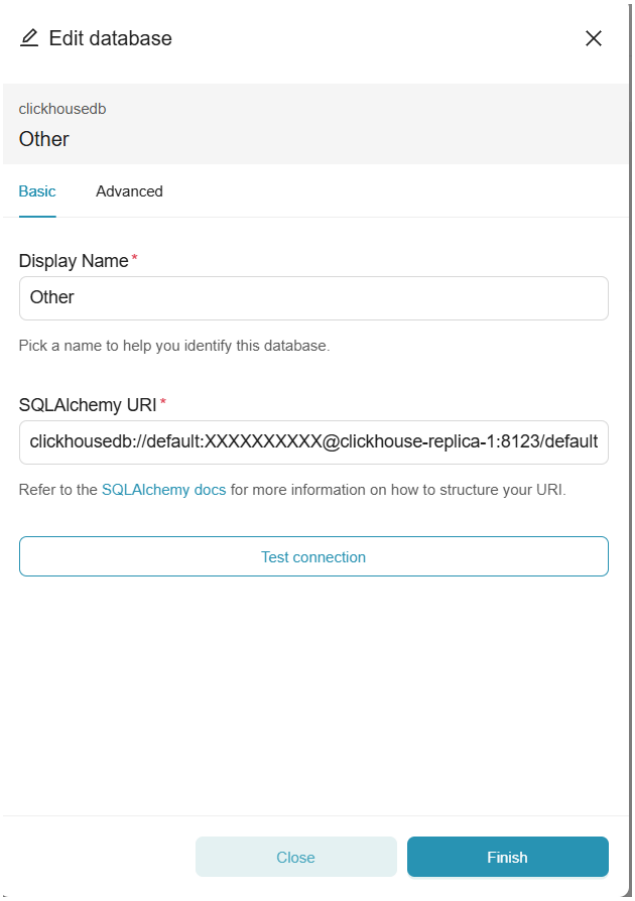
- Установим драйвер (если его нет)

```
docker exec -it superset pip install clickhouse-connect
docker restart superset
```

- Заходим в Superset



- Подключаем ClickHouse как базу данных в Superset



3. Строим дашборд

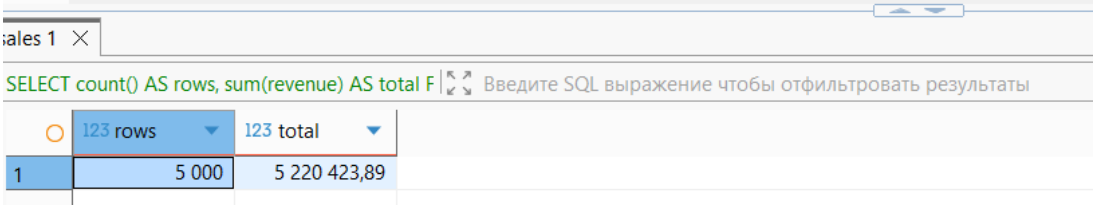
- Подготовим тестовые данные

```sql
-- создаем тестовую БД
CREATE DATABASE IF NOT EXISTS analytics;

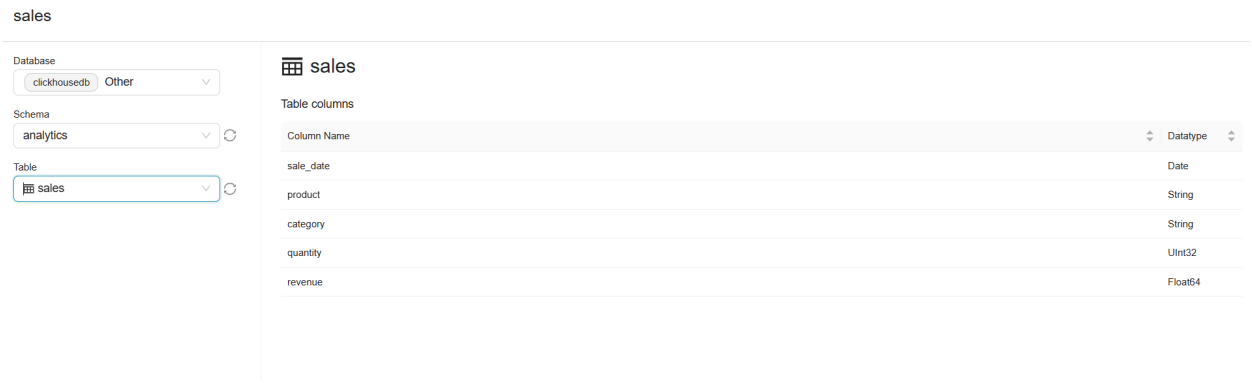
-- создаем таблицу
CREATE TABLE IF NOT EXISTS analytics.sales
(
    sale_date Date,
    product   String,
    category  String,
    quantity  UInt32,
    revenue   Float64
)
ENGINE = MergeTree()
ORDER BY sale_date;

-- заполняем таблицу данными
INSERT INTO analytics.sales
SELECT
    today() - rand() % 90                                        AS sale_date,
    ['Laptop','Phone','Tablet','Monitor','Keyboard'][1 + rand() % 5] AS product,
    ['Electronics','Accessories','Office'][1 + rand() % 3]           AS category,
    1 + rand() % 20                                              AS quantity,
    round(50 + rand() % 2000 + rand() / 4294967295, 2)           AS revenue
FROM numbers(5000);

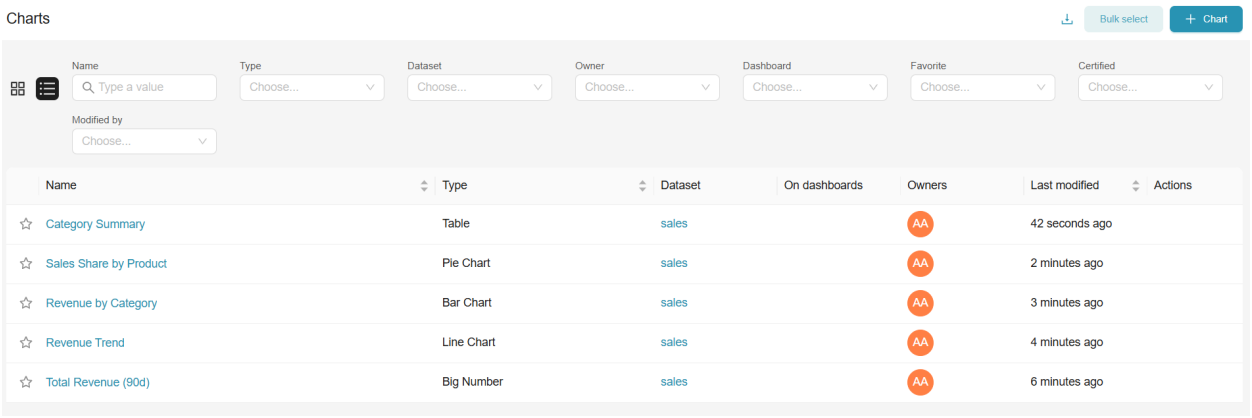
-- проверяем данные в таблице
SELECT count() AS rows, sum(revenue) AS total FROM analytics.sales;
```



- Создаем датасет в Superset



- Создаем визуализации



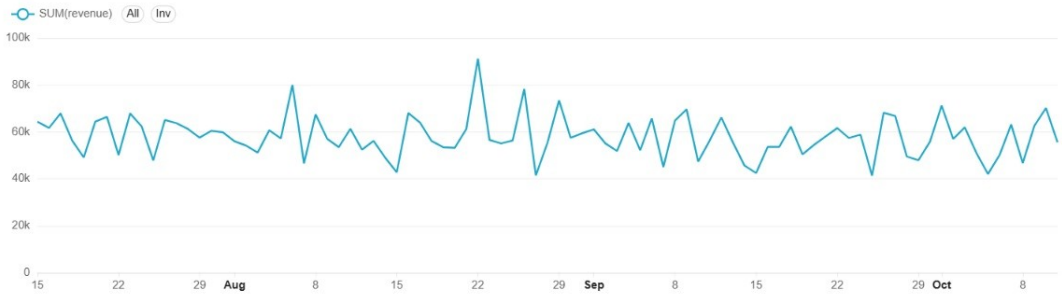
- Создаем дашборд

Total Revenue (90d)

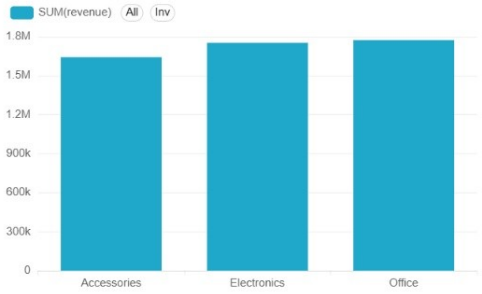
5.17M

Total Revenue (90d)

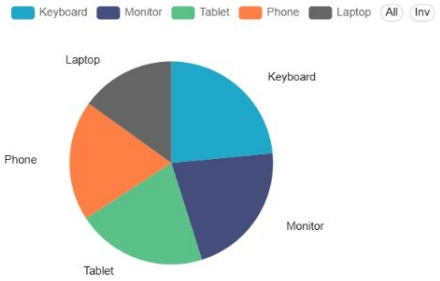
Revenue Trend



Revenue by Category



Sales Share by Product



Category Summary

category	SUM(revenue)	SUM(quantity)	AVG(revenue)
Office	1.77M	17.5k	1.05k
Electronics	1.75M	17.9k	1.05k
Accessories	1.64M	16.4k	1.04k